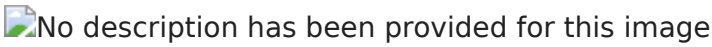
Statistical Learning for Automation Systems

Author: Matteo Cercola

Principal Component Analysis with Python - Case Study

PCA Review

PCA consists in a linear transformation of the available input data and attempts to find out which features mostly explain the variance in the data.



Import Data Analysis and Visualization Libraries

In [1]:
```
%pip install numpy pandas matplotlib seaborn openpyxl
%pip install ipywidgets

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline
```

```
Requirement already satisfied: numpy in /usr/local/python/3.14.2/lib/python
3.14/site-packages (2.5.4)
Requirement already satisfied: pandas in /usr/local/python/3.14.2/lib/python
3.14/site-packages (3.1.0)
Requirement already satisfied: matplotlib in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (3.11.2)
Requirement already satisfied: seaborn in /usr/local/python/3.14.2/lib/pytho
n3.14/site-packages (0.13.2)
Requirement already satisfied: openpyxl in /usr/local/python/3.14.2/lib/pyth
on3.14/site-packages (3.1.5)
Requirement already satisfied: python-dateutil>=2.9.0 in /home/codespace/.lo
cal/lib/python3.14/site-packages (from pandas) (3.0.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/python/3.14.2/
lib/python3.14/site-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/python/3.14.2/lib/
python3.14/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /usr/local/python/3.14.
2/lib/python3.14/site-packages (from matplotlib) (4.68.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/python/3.14.
2/lib/python3.14/site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /home/codespace/.local/li
b/python3.14/site-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in /usr/local/python/3.14.2/lib/pyt
hon3.14/site-packages (from matplotlib) (13.0.0)
Requirement already satisfied: pyparsing>=3 in /usr/local/python/3.14.2/lib/
python3.14/site-packages (from matplotlib) (3.3.3)
Requirement already satisfied: et-xmlfile in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (from openpyxl) (2.0.0)
Requirement already satisfied: six>=1.6 in /home/codespace/.local/lib/python
3.14/site-packages (from python-dateutil>=2.9.0->pandas) (1.17.0)
Requirement already satisfied: tzdata in /home/codespace/.local/lib/python3.
14/site-packages (from python-dateutil>=2.9.0->pandas) (2026.6)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: ipywidgets in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (8.1.9)
Requirement already satisfied: comm>=0.1.3 in /home/codespace/.local/lib/pyt
hon3.14/site-packages (from ipywidgets) (0.2.3)
Requirement already satisfied: ipython>=6.1.0 in /home/codespace/.local/lib/
python3.14/site-packages (from ipywidgets) (9.17.1)
Requirement already satisfied: traitlets>=4.3.1 in /home/codespace/.local/li
b/python3.14/site-packages (from ipywidgets) (5.16.1)
Requirement already satisfied: widgetsnbextension~=4.0.16 in /usr/local/pyth
on/3.14.2/lib/python3.14/site-packages (from ipywidgets) (4.0.16)
Requirement already satisfied: jupyterlab_widgets~=3.0.17 in /usr/local/pyth
on/3.14.2/lib/python3.14/site-packages (from ipywidgets) (3.0.17)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in /home/codes
pace/.local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets)
(1.1.1)
Requirement already satisfied: jedi>=0.18.2 in /home/codespace/.local/lib/py
thon3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.20.1)
Requirement already satisfied: matplotlib-inline>=0.1.6 in /home/codespace/.
local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.2.2)
Requirement already satisfied: pexpect>4.6 in /home/codespace/.local/lib/pyt
hon3.14/site-packages (from ipython>=6.1.0->ipywidgets) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /home/codesp
ace/.local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets)
```

```
(3.0.53)
Requirement already satisfied: psutil>=7 in /home/codespace/.local/lib/pytho
n3.14/site-packages (from ipython>=6.1.0->ipywidgets) (7.2.2)
Requirement already satisfied: pygments>=2.14.0 in /home/codespace/.local/li
b/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (2.21.0)
Requirement already satisfied: stack_data>=0.6.0 in /home/codespace/.local/l
ib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.6.3)
Requirement already satisfied: wcwidth>=0.1.4 in /home/codespace/.local/lib/
python3.14/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=6.1.0
->ipywidgets) (0.9.4)
Requirement already satisfied: parso<0.9.0,>=0.8.7 in /home/codespace/.loca
l/lib/python3.14/site-packages (from jedi>=0.18.2->ipython>=6.1.0->ipywidget
s) (0.8.7)
Requirement already satisfied: ptyprocess>=0.5 in /home/codespace/.local/li
b/python3.14/site-packages (from pexpect>4.6->ipython>=6.1.0->ipywidgets)
(0.7.0)
Requirement already satisfied: executing>=1.2.0 in /home/codespace/.local/li
b/python3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidge
ts) (2.3.0)
Requirement already satisfied: asttokens>=2.1.0 in /home/codespace/.local/li
b/python3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidge
ts) (3.0.2)
Requirement already satisfied: pure-eval in /home/codespace/.local/lib/pytho
n3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (0.
2.4)
Note: you may need to restart the kernel to use updated packages.
```

Import Dataset

We'll work with the built-in Breast Cancer dataset from Scikit Learn since it has a very high number of features.

In [2]:
```
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()                       # Assign breast cancer d
cancer                                              # 'cancer' is a variable
```

Out[2]:

```
{'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
         1.189e-01],
        [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
         8.902e-02],
        [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
         8.758e-02],
        ...,
        [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
         7.820e-02],
        [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
         1.240e-01],
        [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
         7.039e-02]], shape=(569, 30)),
 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
        1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
        1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
        1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
        0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
        1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
        0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
        1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
        0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
        1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
        1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
        1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
        1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
        1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1]),
 'frame': None,
 'target_names': array(['malignant', 'benign'], dtype='<U9'),
 'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer Wisconsin (diagnosti
c) dataset\n--------------------------------------------\n\n**Data Set Char
acteristics:**\n\n:Number of Instances: 569\n\n:Number of Attributes: 30 nu
meric, predictive attributes and the class\n\n:Attribute Information:\n
- radius (mean of distances from center to points on the perimeter)\n    -
texture (standard deviation of gray-scale values)\n    - perimeter\n    - a
rea\n    - smoothness (local variation in radius lengths)\n    - compactnes
s (perimeter^2 / area - 1.0)\n    - concavity (severity of concave portions
of the contour)\n    - concave points (number of concave portions of the co
ntour)\n    - symmetry\n    - fractal dimension ("coastline approximation"
- 1)\n\n    The mean, standard error, and "worst" or largest (mean of the t
hree\n    worst/largest values) of these features were computed for each im
age,\n    resulting in 30 features.  For instance, field 0 is Mean Radius,
```

field\n   10 is Radius SE, field 20 is Worst Radius.\n\n   - class:\n
- WDBC-Malignant\n   - WDBC-Benign\n\n:Summary Statistics:\n\n====
================================ ====== ======\n
Min   Max\n==================================== ====== ======\nradius (me
an):   6.981 28.11\ntexture (mean):
9.71   39.28\nperimeter (mean):   43.79 188.5\narea (mea
n):   143.5 2501.0\nsmoothness (mean):
0.053 0.163\ncompactness (mean):   0.019 0.345\nconcavity
(mean):   0.0   0.427\nconcave points (mean):
0.0   0.201\nsymmetry (mean):   0.106 0.304\nfractal d
imension (mean):   0.05   0.097\nradius (standard error):
0.112 2.873\ntexture (standard error):   0.36   4.885\nperimeter
(standard error):   0.757 21.98\narea (standard error):
6.802 542.2\nsmoothness (standard error):   0.002 0.031\ncompactne
ss (standard error):   0.002 0.135\nconcavity (standard error):
0.0   0.396\nconcave points (standard error):   0.0   0.053\nsymmetry
(standard error):   0.008 0.079\nfractal dimension (standard erro
r):   0.001 0.03\nradius (worst):   7.93   36.04\ntext
ure (worst):   12.02 49.54\nperimeter (worst):
50.41 251.2\narea (worst):   185.2 4254.0\nsmoothne
ss (worst):   0.071 0.223\ncompactness (worst):
0.027 1.058\nconcavity (worst):   0.0   1.252\nconcave p
oints (worst):   0.0   0.291\nsymmetry (worst):
0.156 0.664\nfractal dimension (worst):   0.055 0.208\n=========
============================ ====== ======\n\n:Missing Attribute Values: No
ne\n\n:Class Distribution: 212 - Malignant, 357 - Benign\n\n:Creator: Dr.
William H. Wolberg, W. Nick Street, Olvi L. Mangasarian\n\n:Donor: Nick Str
eet\n\n:Date: November, 1995\n\nThis is a copy of UCI ML Breast Cancer Wisc
onsin (Diagnostic) datasets.\nhttps://goo.gl/U2Uwz2\n\nFeatures are compute
d from a digitized image of a fine needle\naspirate (FNA) of a breast mass.
They describe\ncharacteristics of the cell nuclei present in the image.\n\n
Separating plane described above was obtained using\nMultisurface Method-Tr
ee (MSM-T) [K. P. Bennett, "Decision Tree\nConstruction Via Linear Programm
ing." Proceedings of the 4th\nMidwest Artificial Intelligence and Cognitive
Science Society,\npp. 97-101, 1992], a classification method which uses lin
ear\nprogramming to construct a decision tree. Relevant features\nwere sel
ected using an exhaustive search in the space of 1-4\nfeatures and 1-3 sepa
rating planes.\n\nThe actual linear program used to obtain the separating p
lane\nin the 3-dimensional space is that described in:\n[K. P. Bennett and
O. L. Mangasarian: "Robust Linear\nProgramming Discrimination of Two Linear
ly Inseparable Sets",\nOptimization Methods and Software 1, 1992, 23-34].\n
\nThis database is also available through the UW CS ftp server:\n\nftp ftp.
cs.wisc.edu\ncd math-prog/cpo-dataset/machine-learn/WDBC/\n\n.. dropdown::
References\n\n - W.N. Street, W.H. Wolberg and O.L. Mangasarian. Nuclear f
eature extraction\n   for breast tumor diagnosis. IS&T/SPIE 1993 Internati
onal Symposium on\n   Electronic Imaging: Science and Technology, volume 1
905, pages 861-870,\n   San Jose, CA, 1993.\n - O.L. Mangasarian, W.N. St
reet and W.H. Wolberg. Breast cancer diagnosis and\n   prognosis via linea
r programming. Operations Research, 43(4), pages 570-577,\n   July-August
1995.\n - W.H. Wolberg, W.N. Street, and O.L. Mangasarian. Machine learnin
g techniques\n   to diagnose breast cancer from fine-needle aspirates. Can
cer Letters 77 (1994)\n   163-171.\n',
 'feature_names': array(['mean radius', 'mean texture', 'mean perimeter',
'mean area',
        'mean smoothness', 'mean compactness', 'mean concavity',
        'mean concave points', 'mean symmetry', 'mean fractal dimension',

```
        'radius error', 'texture error', 'perimeter error', 'area error',
        'smoothness error', 'compactness error', 'concavity error',
        'concave points error', 'symmetry error',
        'fractal dimension error', 'worst radius', 'worst texture',
        'worst perimeter', 'worst area', 'worst smoothness',
        'worst compactness', 'worst concavity', 'worst concave points',
        'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
 'filename': 'breast_cancer.csv',
 'data_module': 'sklearn.datasets.data'}
```

In [3]:
```
# Look at keys of dictionary
cancer.keys()
```

Out[3]:
```
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_nam
es', 'filename', 'data_module'])
```

In [4]:
```
# Inspect a field
# print(cancer['DESCR'])

# Inspect output
cancer['target_names']

# Inspect output value
cancer['target']

# Generate DataFrame
df = pd.DataFrame(cancer['data'], columns = cancer['feature_names'])
df.head()
```

Out[4]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	c
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	

5 rows × 30 columns

In [5]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 30 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   mean radius              569 non-null    float64
 1   mean texture             569 non-null    float64
 2   mean perimeter           569 non-null    float64
 3   mean area                569 non-null    float64
 4   mean smoothness          569 non-null    float64
 5   mean compactness         569 non-null    float64
 6   mean concavity           569 non-null    float64
 7   mean concave points      569 non-null    float64
 8   mean symmetry            569 non-null    float64
 9   mean fractal dimension   569 non-null    float64
 10  radius error             569 non-null    float64
 11  texture error            569 non-null    float64
 12  perimeter error          569 non-null    float64
 13  area error               569 non-null    float64
 14  smoothness error         569 non-null    float64
 15  compactness error        569 non-null    float64
 16  concavity error          569 non-null    float64
 17  concave points error     569 non-null    float64
 18  symmetry error           569 non-null    float64
 19  fractal dimension error  569 non-null    float64
 20  worst radius             569 non-null    float64
 21  worst texture            569 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               569 non-null    float64
 24  worst smoothness         569 non-null    float64
 25  worst compactness        569 non-null    float64
 26  worst concavity          569 non-null    float64
 27  worst concave points     569 non-null    float64
 28  worst symmetry           569 non-null    float64
 29  worst fractal dimension  569 non-null    float64
dtypes: float64(30)
memory usage: 133.5 KB
```

In [6]:
```
df.describe()
```

Out[6]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	m compact
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.00
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.10
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.05
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.01
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.06
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.09
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.13
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.34

8 rows × 30 columns

Principal Component Analysis

As intuitive, it is difficult to visualize high dimensional data, so we can use PCA for dimensionality reduction.

Before doing this, we need to remove bias and scale our data so that each feature has unit variance.

Data Pre-Processing

Scikit-Learn function named StandardScaler() standardizes features by removing the bias and scaling to unit variance (it automatically performs input centering and input normalization).

In [7]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(df)
scaled_data = scaler.transform(df)
```

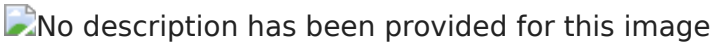
Singular Value Decomposition

SVD takes the data and decomposes it into 3 matrices s.t.

$$X = U\Sigma V^*$$

Σ is diagonal (rectangular) with dimensions $m \times n$ and each diagonal element represent the direction of maximum "stretching"

In simple terms, SVD decomposes the transformation induced by X into two rotations U and V^* and a scaling Σ


No description has been provided for this image

In [8]:
```
# Compute SVD for scaled_data matrix, obtaining as output the matrices for i
U, sigma, VT = np.linalg.svd(scaled_data, full_matrices=True)

# sigma is an array of dimension 30 containing the ordered singular values c
print(sigma)
print('Size of sigma', sigma.shape)

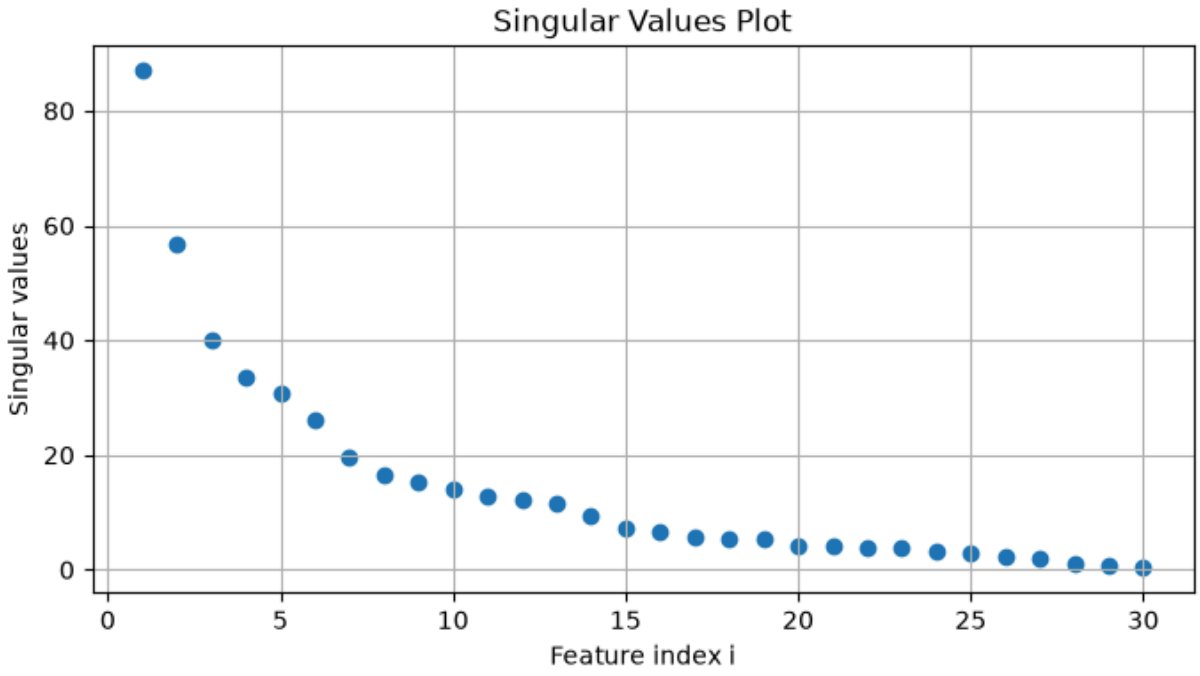
# Sigma is the diagonal matrix of dimension 30 * 30 with ordered singular va
Sigma = np.diag(sigma)           # Diagonal matrix
Sigma.shape
```

```
[86.93235745 56.90677266 40.04263937 33.57058877 30.62887007 26.2104161
 19.60102663 16.46800391 15.4017255  14.12602481 12.93205441 12.19019359
 11.71889139  9.4519063   7.31866075  6.74106325  5.81360925  5.47175363
  5.30591648  4.21066503  4.12971871  3.95133143  3.72155031  3.20519874
  2.96796958  2.15709923  1.98150547  0.95096438  0.65273958  0.27514088]
Size of sigma (30,)
```

Out[8]: (30, 30)

In [9]:
```
# Plot ordered singular values
plt.figure(figsize = (8,4))
plt.scatter(range(1, len(sigma)+1), sigma)
plt.grid()
plt.ylabel('Singular values')
plt.xlabel('Feature index i')
plt.title('Singular Values Plot')
```

Out[9]: Text(0.5, 1.0, 'Singular Values Plot')



Most of the information is contained in the first two principal components. We can use PCA to find the first two principal components and visualize the data in this new, two-dimensional space, with a single scatter plot.

Note that the columns of V are the principal directions (eigenvectors), so they indicate the most important directions for the data distribution. Thus, to compress the data, we should project X on the columns of V

In [10]:
```
# Project scaled_data into the first two principal components
X_reduced = np.dot(scaled_data, VT.T[:,:2])     # We are computing X * V alo
X_reduced.shape

Sigma.shape
```

Out[10]:
```
(30, 30)
```

PCA Algorithm

Instantiate a PCA object, find the principal components using the fit() method, then apply the rotation and dimensionality reduction by calling transform().

It is also possible to specify how many components we want to keep when creating the PCA object.

In [11]:
```
from sklearn.decomposition import PCA                    # PCA() import
pca = PCA(n_components = 2)                              # PCA() object instant
pca.fit(scaled_data)                                     # fit(X) method fits t
```

Out[11]:

▾ PCA ⓘ ?

▸ Parameters

▸ Fitted attributes

▸ 2 features

Now we can transform data to the first 2 principal components.

In [12]:
```
# transform(X) method applies dimensionality reduction to X
x_pca = pca.transform(scaled_data)
print(scaled_data.shape)                                 # Dimensions o
print(x_pca.shape)                                       # Dimensions o
```

```
(569, 30)
(569, 2)
```

In [13]:
```
x_pca_df = pd.DataFrame(x_pca, columns=['PC1', 'PC2'])        # Pandas Dataf
x_pca_df.head()
```

Out[13]:

	PC1	PC2
0	9.192837	1.948583
1	2.387802	-3.768172
2	5.733896	-1.075174
3	7.122953	10.275589
4	3.935302	-1.948072

In [26]:
```
x_pca_df.info()
x_pca_df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   PC1     569 non-null    float64
 1   PC2     569 non-null    float64
dtypes: float64(2)
memory usage: 9.0 KB
```

Out[26]:

	PC1	PC2
count	5.690000e+02	5.690000e+02
mean	-4.995028e-17	-6.243785e-18
std	3.647601e+00	2.387755e+00
min	-5.565736e+00	-7.775853e+00
25%	-2.757430e+00	-1.499825e+00
50%	-1.173831e+00	-1.959558e-01
75%	2.508766e+00	1.228297e+00
max	1.631923e+01	1.257319e+01

Observation: We've reduced 30 dimensions to just 2! Let's plot these two dimensions out!

Plot Data in the Reduced Feature Space

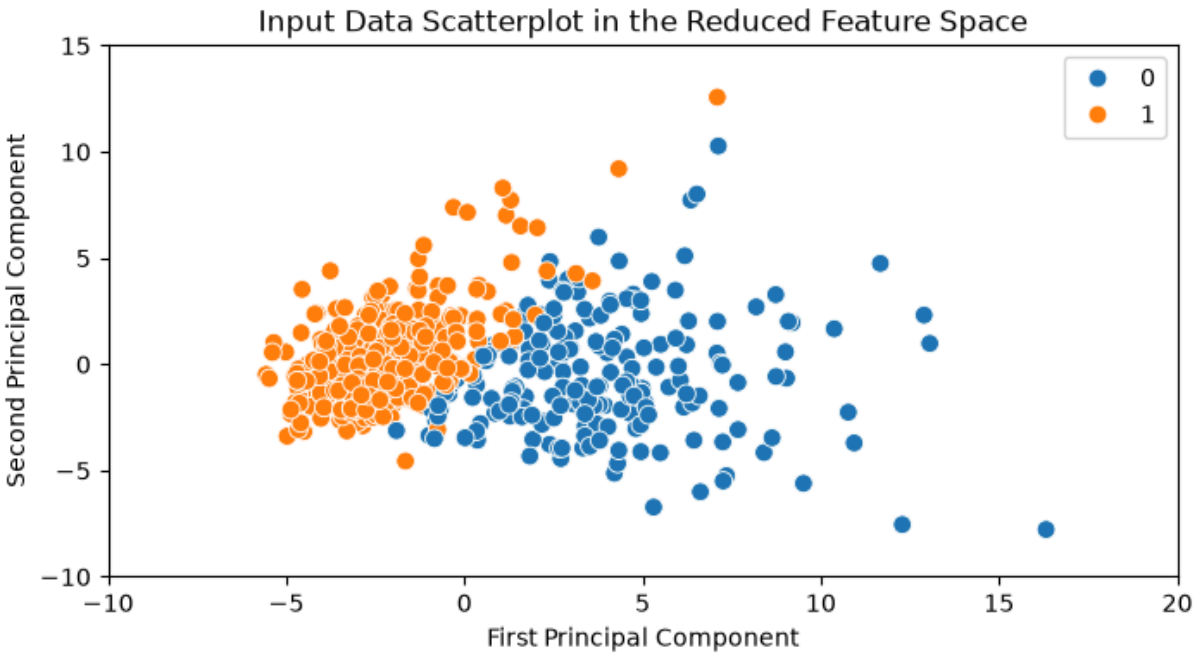
In [15]:
```
plt.figure(figsize=(8,4))

sns.scatterplot(x=x_pca[:, 0], y=x_pca[:, 1], hue=cancer['target'], s=60)


plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('Input Data Scatterplot in the Reduced Feature Space')
```

```
plt.xlim([-10, 20])
plt.ylim([-10, 15])
```

Out[15]: (-10.0, 15.0)



By using these two principal components, we can easily separate the two classes (target = 0, target = 1).

Interpreting the components

Unfortunately, with this great power of dimensionality reduction, comes the cost of being able to easily understand what these components represent.

The components correspond to linear combinations of the original features. The components themselves are stored as an attribute of the fitted PCA object:

In [16]:
```
pca.components_
```

Out[16]:
```
array([[ 0.21890244,  0.10372458,  0.22753729,  0.22099499,  0.14258969,
         0.23928535,  0.25840048,  0.26085376,  0.13816696,  0.06436335,
         0.20597878,  0.01742803,  0.21132592,  0.20286964,  0.01453145,
         0.17039345,  0.15358979,  0.1834174 ,  0.04249842,  0.10256832,
         0.22799663,  0.10446933,  0.23663968,  0.22487053,  0.12795256,
         0.21009588,  0.22876753,  0.25088597,  0.12290456,  0.13178394],
       [-0.23385713, -0.05970609, -0.21518136, -0.23107671,  0.18611302,
         0.15189161,  0.06016536, -0.0347675 ,  0.19034877,  0.36657547,
        -0.10555215,  0.08997968, -0.08945723, -0.15229263,  0.20443045,
         0.2327159 ,  0.19720728,  0.13032156,  0.183848  ,  0.28009203,
        -0.21986638, -0.0454673 , -0.19987843, -0.21935186,  0.17230435,
         0.14359317,  0.09796411, -0.00825724,  0.14188335,  0.27533947]])
```

In this numpy matrix array, each row represents a principal component and each column relates back to the original features.

We can visualize this relationship with a heatmap.

In [17]:
```
df_comp = pd.DataFrame(pca.components_ , columns = cancer['feature_names'])
df_comp                                                    # Show df_comp
```

Out[17]:

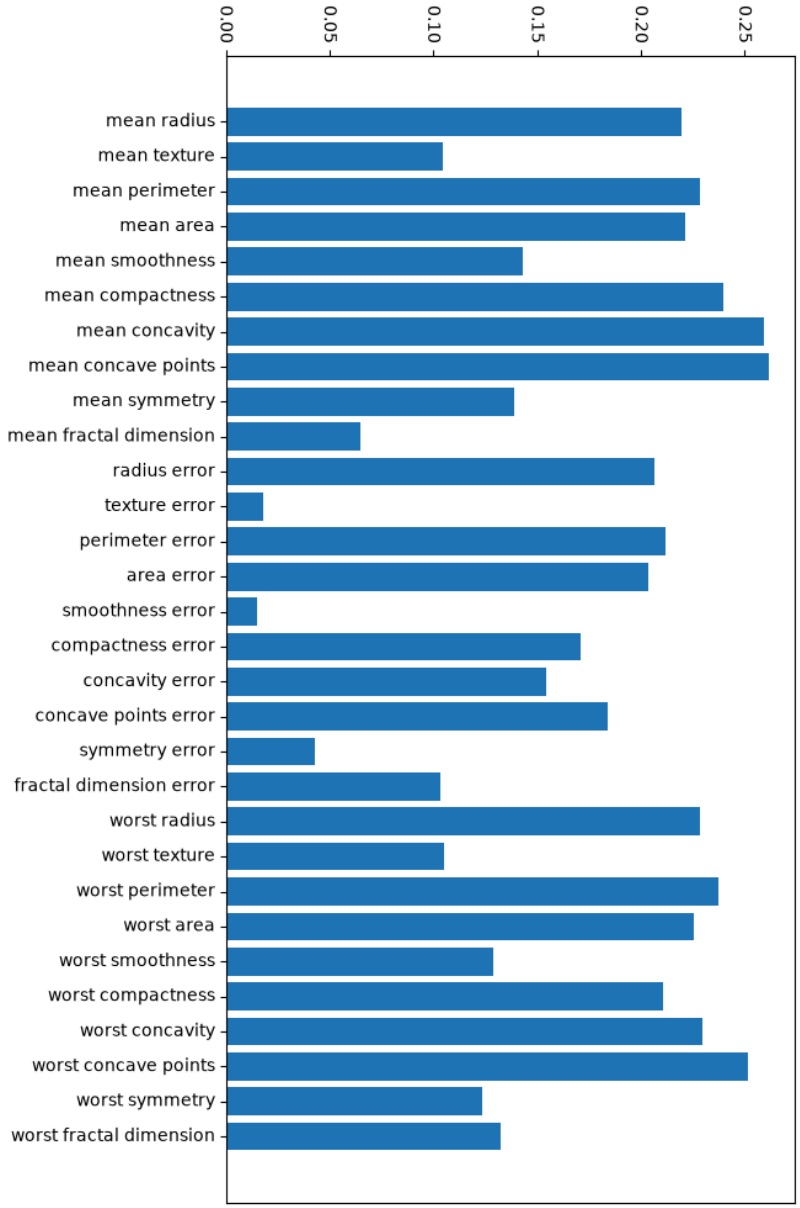
	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	r conc
0	0.218902	0.103725	0.227537	0.220995	0.142590	0.239285	0.25
1	-0.233857	-0.059706	-0.215181	-0.231077	0.186113	0.151892	0.06

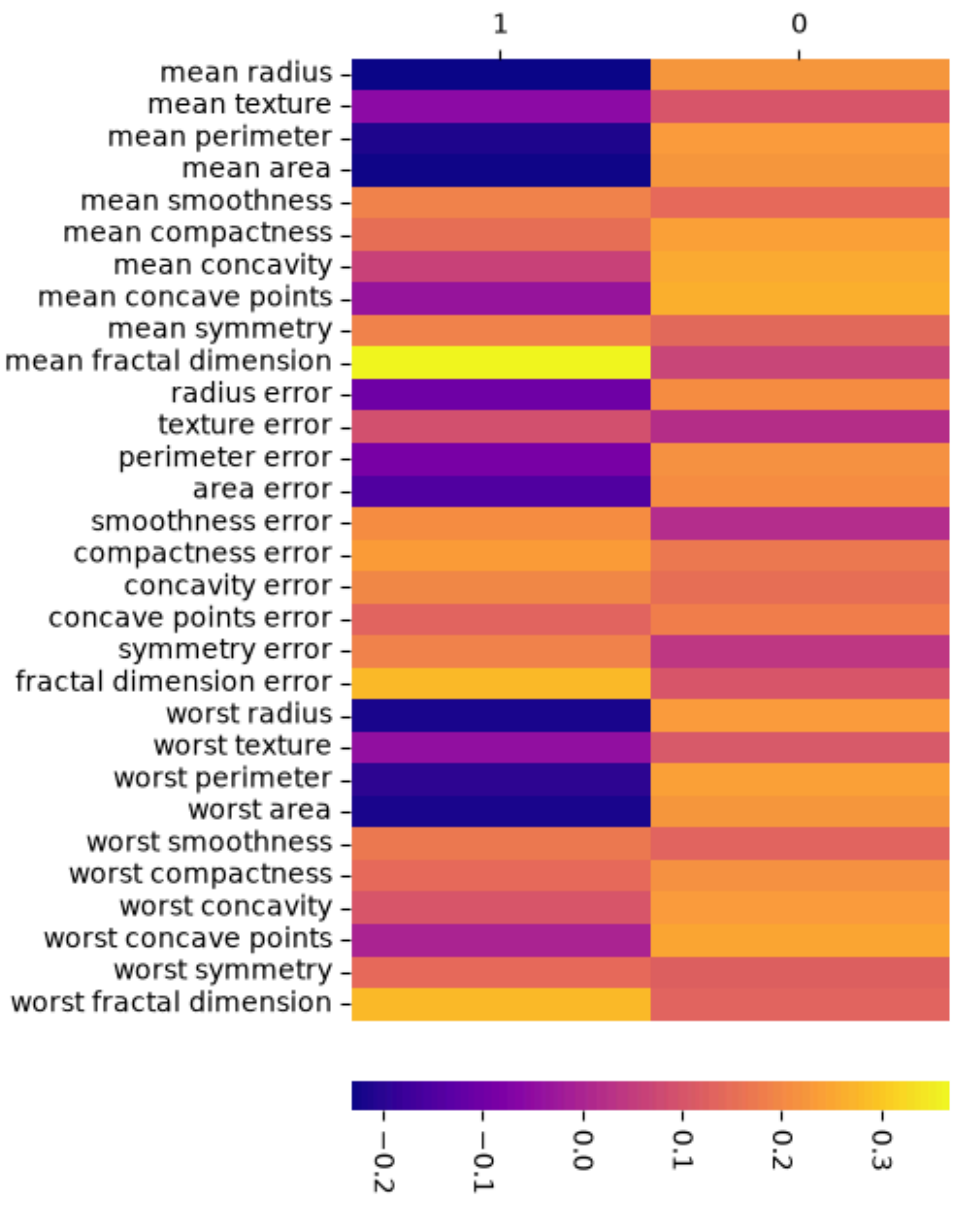
2 rows × 30 columns

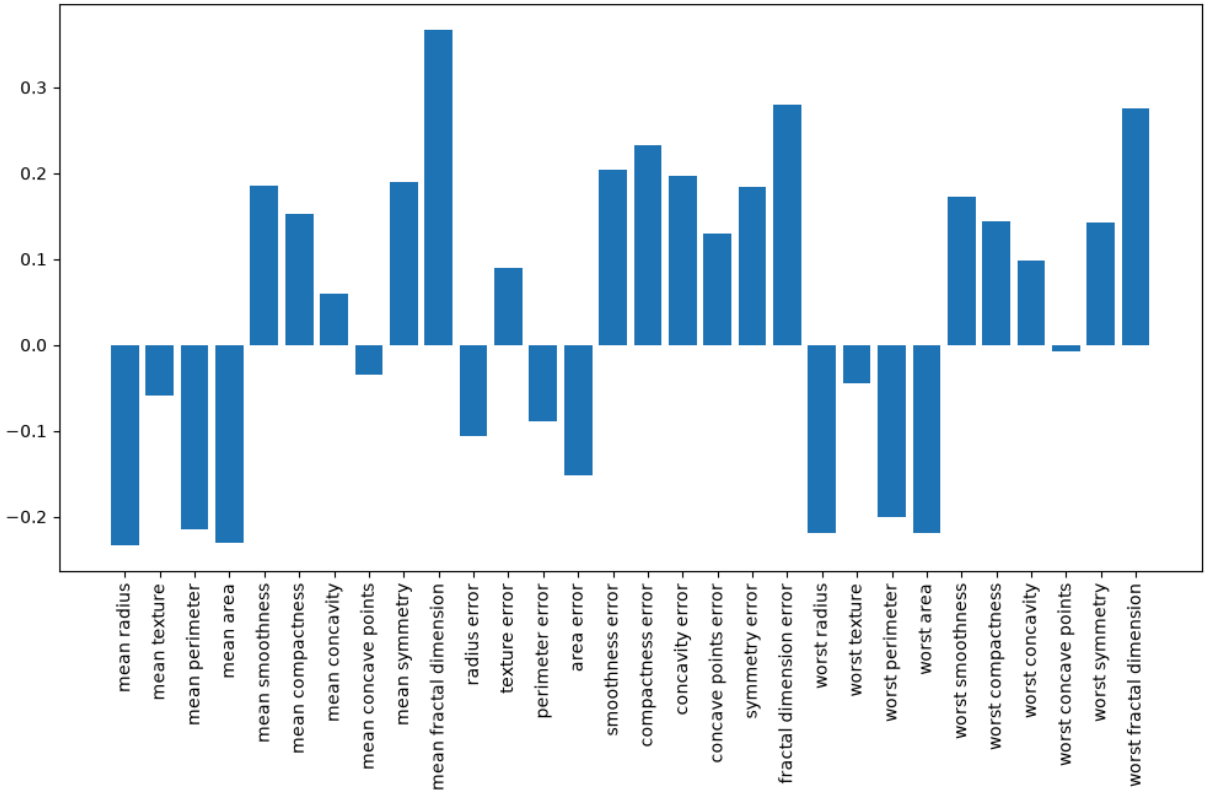
In [18]:
```
# Heatmap Plot
plt.figure(figsize=(8,4))
sns.heatmap(df_comp , cmap='plasma')

# Coefficients of first direction
plt.figure(figsize=(12,6))
plt.bar(np.arange(len(df_comp.iloc[0,:])), df_comp.iloc[0,:])
plt.xticks(np.arange(len(df_comp.iloc[0,:])), df_comp.columns, rotation=90)
plt.show()

# Coefficients of second direction
plt.figure(figsize=(12,6))
plt.bar(np.arange(len(df_comp.iloc[1,:])), df_comp.iloc[1,:])
plt.xticks(np.arange(len(df_comp.iloc[1,:])), df_comp.columns, rotation=90)
plt.show()
```


0.00
0.05
0.10
0.15
0.20
0.25
mean radius
mean texture
mean perimeter
mean area
mean smoothness
mean compactness
mean concavity
mean concave points
mean symmetry
mean fractal dimension
radius error
texture error
perimeter error
area error
smoothness error
compactness error
concavity error
concave points error
symmetry error
fractal dimension error
worst radius
worst texture
worst perimeter
worst area
worst smoothness
worst compactness
worst concavity
worst concave points
worst symmetry
worst fractal dimension


1
0
mean radius
mean texture
mean perimeter
mean area
mean smoothness
mean compactness
mean concavity
mean concave points
mean symmetry
mean fractal dimension
radius error
texture error
perimeter error
area error
smoothness error
compactness error
concavity error
concave points error
symmetry error
fractal dimension error
worst radius
worst texture
worst perimeter
worst area
worst smoothness
worst compactness
worst concavity
worst concave points
worst symmetry
worst fractal dimension
−0.2
−0.1
0.0
0.1
0.2
0.3



This heatmap and the color bar basically represent the correlation between the various features and the principal component itself.

Can we use PCA in a way other than simply visualizing higher-dimensional data?

We can use PCA to reduce the size of our model to simplify its structure without loosing excessive information!

When you perform Principal Component Analysis (PCA), the goal is to find new orthogonal directions (principal components) that capture as much of the total variance in your dataset as possible.

Each principal component i has an associated variance (also called an eigenvalue of the covariance matrix), denoted by λ_i and correspond to the square of the principal components

In [19]:
```
# Transform the data into the new space
pca_result = np.dot(scaled_data, VT.T)

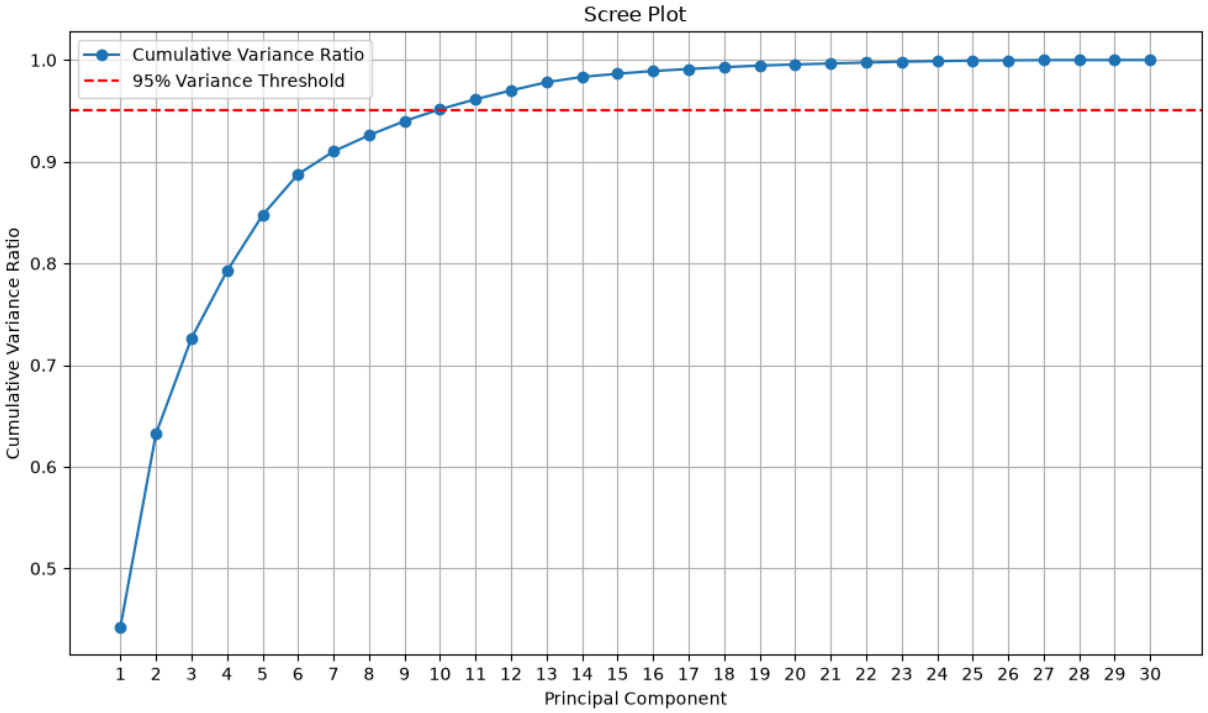
# Compute the explained variance
explained_variance = (sigma ** 2) / (scaled_data.shape[1] - 1)  # Variance e

# Compute total variance
total_variance = np.sum(explained_variance)                     # Total vari
```

```
# Compute explained variance ratio
explained_variance_ratio = explained_variance / total_variance # Proportion

# Compute cumulative variance ratio
cumulative_variance_ratio = np.cumsum(explained_variance_ratio) # Cumulative

#Create the scree plot
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(explained_variance) + 1), cumulative_variance_ratio, m
plt.axhline(y=0.95, color='red', linestyle='--', label='95% Variance Thresho
plt.title('Scree Plot')
plt.xlabel('Principal Component')
plt.ylabel('Cumulative Variance Ratio')
plt.xticks(range(1, len(explained_variance) + 1))
plt.grid()
plt.legend()
plt.tight_layout()
plt.show()
```



In [20]:
```
K = np.argmax(cumulative_variance_ratio >= 0.95) + 1
K # Number of principal components required to retain at least 95% of the va
```

Out[20]:
```
np.int64(10)
```

This plots is useful to have an heuristic to identify the K value for the PCA

In [21]:
```
X_reduced = np.dot(scaled_data, VT.T[:,:K])          # Project scaled_data i
X_reduced.shape
```

Out[21]:
```
(569, 10)
```

In [22]:
```
from sklearn.model_selection import train_test_split
```

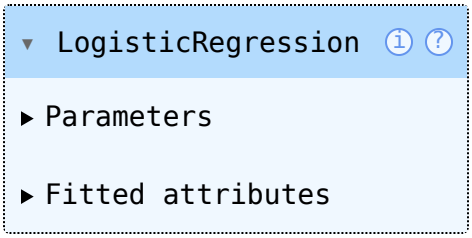
```
X_train, X_test, y_train, y_test = train_test_split(X_reduced, cancer['targe
X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

Out[22]: ((455, 10), (455,), (114, 10), (114,))

```
In [23]: from sklearn.linear_model import LogisticRegression                    # 1

In [24]: logmodel = LogisticRegression()                                        # L
         logmodel.fit(X_train, y_train)                                         # M
```

Out[24]:




```
In [25]: from sklearn.metrics import confusion_matrix, classification_report

         predictions = logmodel.predict(X_test)                                 # P
         print(classification_report(y_test, predictions))                      # C
```

```
              precision    recall  f1-score   support

           0       0.98      0.98      0.98        43
           1       0.99      0.99      0.99        71

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

What was the accuracy of the NN model? Again 98%

Conclusion

The presented case study is an example of application of PCA to a dataset characterized by a very high-dimensional feature space.

- First, by projecting high-dimensional data onto the first two principal components, PCA provided an effective 2D representation that preserved most of the variance, allowing us to visually identify clusters, trends, and potential outliers in the dataset.
- Second, we applied PCA as a preprocessing step prior to training a machine learning model, showing that reducing the number of features can significantly decrease model complexity and computational cost while maintaining comparable predictive performance.

Overall, the study highlights how PCA helps uncover the underlying structure of high-dimensional data and improves model efficiency, making it a fundamental

tool in exploratory data analysis and modern data-driven pipelines.

The drawback relates to the poor interpretability of what principal components represent.